Отчёт по домашнему заданию

Курс: Распределённые системы и сетевые протоколы

Тема: Стандарты форматирования сообщений
Студент: [Имя Фамилия]
Дата: Октябрь 2026 г.

1. Анализ формата сообщений в экосистеме маркетплейса (на примере Ozon)

В качестве объекта исследования выбрана масштабная распределённая e-commerce платформа **Ozon**. Система обслуживает миллионы активных пользователей и продавцов, обрабатывая десятки тысяч заказов в минуту. Экосистема маркетплейса делится на несколько ключевых зон взаимодействия, каждая из которых выдвигает специфические требования к сериализации данных.

> **Архитектурный профиль передачи данных:**
> - **Client-to-Backend (Публичный Seller API & Мобильное приложение):** Формат `JSON` поверх HTTPS/REST.
> - **Inter-service Communication (Внутренние микросервисы):** Формат `Protocol Buffers (Protobuf)` поверх gRPC.
> - **Event Streaming (Асинхронная шина событий):** Формат `Apache Avro` / `Protobuf` в Apache Kafka.

2. Обоснование архитектурного выбора и альтернативы

Выбор форматов в Ozon обусловлен разделением зон ответственности и спецификой нагрузок:

- **JSON для публичных REST API:** Обеспечивает мгновенный порог входа для внешних разработчиков и интеграторов (учётных систем 1С, МойСклад). JSON является общепринятым стандартом Web API, легко отлаживается через cURL/Postman и native-инспектируется браузерами.
- **Protobuf для микросервисной архитектуры (gRPC):** Высоконагруженная сеть микросервисов (поиск, корзина, процессинг платежей) требует минимальной задержки (latency) и высокой плотности упаковки. Protobuf снижает расходы на сетевой I/O и нагрузку на CPU при маршалинге. Строгая типизация через `.proto` контракты предотвращает ошибки интеграции.
- **Avro для Event Streaming (Kafka):** Обеспечивает компактную сериализацию без дублирования схемы в каждом событии (схема хранится отдельно в Schema Registry), что критично при сохранении петабайт логов и событий.

Неподходящие альтернативы:

- **XML:** Избыточен по объёму (теги занимают до 60% размера), требует затратного DOM/SAX-парсинга. В условиях highload увеличивает чеки за сетевой трафик и серверные мощности.
- **YAML:** Медленный парсинг (до 10 раз медленнее JSON), чувствительность к отступам создаёт риски падающих парсеров при сетевой передаче. Подходит для конфигураций, но не для messaging.
- **CBOR / MessagePack:** Хотя они компактны и бессхемны, отсутствие жесткого контракта (в отличие от Protobuf) усложняет командную разработку сотен микросервисов.

3. Сравнительный анализ представлений данных (Объект «Заказ»)

Ниже представлен пример структуры данных заказа пользователя в трёх различных формата сериализации.

1. Текстовое представление JSON

```
{
  "order_id": 1089452,
  "customer_id": "usr_9941",
  "status": "PAID",
  "items": [
    { "product_id": 402, "name": "Беспроводная мышь", "price": 1290.00, "qty": 1 },
    { "product_id": 881, "name": "Коврик для мыши", "price": 450.50, "qty": 2 }
  ],
  "total_amount": 2191.00
}
```

2. Текстовое представление XML

```
<?xml version="1.0" encoding="UTF-8"?>
<order id="1089452" status="PAID">
  <customer_id>usr_9941</customer_id>
  <items>
    <item product_id="402"><name>Беспроводная мышь</name><price>1290.00</price><qty>1</qty></
item>
    <item product_id="881"><name>Коврик для мыши</name><price>450.50</price><qty>2</qty></ite
m>
  </items>
  <total_amount>2191.00</total_amount>
</order>
```

3. Схема Protocol Buffers (order.proto)

```
syntax = "proto3";
package ecommerce;

message OrderItem {
  int64 product_id = 1;
  string name = 2;
  double price = 3;
  int32 qty = 4;
}

message Order {
  int64 order_id = 1;
  string customer_id = 2;
  enum Status { UNKNOWN = 0; PAID = 1; SHIPPED = 2; }
  Status status = 3;
  repeated OrderItem items = 4;
  double total_amount = 5;
}
```

Сводное сравнение метрик и характеристик

Параметр / Критерий	JSON	XML	Protocol Buffers
Примерный размер	~260 байт	~380 байт	**~78 байт** (~70% экономии)

4. Эффекты и ограничения перехода с JSON на Protobuf

Гипотетическая миграция внутренних сервисов приложения с JSON на Protobuf несёт следующие системные последствия:

Выигрыш и преимущества:

- **Сокращение сетевого трафика на 60–75%:** За счёт отбрасывания имен ключей и кодирования чисел в компактные Varint-форматы.
- **Снижение CPU-нагрузки и Latency:** Десериализация бинарного Protobuf в 3–5 раз быстрее парсинга строкового JSON. Уменьшается пауза сборщика мусора (GC) за счёт генерации строго структурированного DTO-кода.
- **Автоматическая генерация SDK:** Кодогенератор `protoc` создает клиентские биндинги для Go, Java, Python, TypeScript из одного `.proto` файла.
- **Гарантированная обратная/прямая совместимость:** Правило тегов полей гарантирует, что старые сервисы просто проигнорируют новые поля, не вызывая Fatal Failure.

Возникающие ограничения и риски:

- **Сложность отладки (Loss of Debuggability):** Сетевой дампер (Wireshark, Charles, Chrome DevTools) показывает «мусор» вместо текста. Требуются специализированные gRPC-клиенты (BloomRPC, gRPCurl) и наличия `.proto` файла под рукой.
- **Несовместимость с веб-браузерами:** Браузеры не могут напрямую работать с HTTP/2 gRPC бинарными кадрами без прокси-слоя (gRPC-Web или Envoy Proxy).
- **Жесткость процесса сборки:** Любое изменение требует единого репозитория схем (Schema Registry) или Git submodule и этапа CI/CD кодогенерации.

5. Целесообразность использования XML в современных системах

Несмотря на уход XML из веб-разработки, существует класс систем, где XML остается **единственно верным и оптимальным решением**.

> **Кейс: Межбанковские финансовые транзакции и Гос. реестры (ISO 20022 / СБП / ДБО)**
>
> В банковском секторе и системах электронного документооборота (ЭДО) стандартом является семейство форматов **ISO 20022** и **XML-DSig**.
>
> - **Криптографическая подпись (XML-DSig):** Стандарт XML позволяет подписывать ЭЦП как весь документ, так и его отдельные фрагменты (ноды) с канонизацией (C14N). В JSON аналогичные стандарты (JWS/JWE) значительно менее зрелы.
> - **Строгая валидация XSD (XML Schema Definition):** Межбанковский перевод требует стопроцентной проверки типов, диапазонов и структур данных на этапе входа в шлюз, до выполнения кода приложения.
> - **Пространства имён (Namespaces):** Позволяют безопасно объединять в одном документе данные из разных ведомств (Налоговая, Банк, Таможня), исключая коллизии имен полей.